Universidade Estadual da Paraíba — UEPB

Centro de Ciências Exatas e Sociais Aplicadas — CCEA

Curso de Bacharelado em Ciência da Computação

Estrutura de Dados

Análise de Algoritmos

Heron Aragão Monteiro

Slides da disciplina baseados no material do professor **Janderson J. B. Aguiar** (UEPB/CCEA)

Estrutura de Dados
Análise de Algoritmos

- Objetivos
  - Considerar todas as entradas
    - Abstrair detalhes
      - Analisar algoritmos independentemente de hardware e software
      - Análise feita em alto nível (pseudocódigo)
        - Avaliar sem precisar rodar experimentos

```
public boolean XYZ(int n, ...) {
    ...
    ...
}
```

Algoritmos

Análises

$f(n)$

Matemática

Estrutura de Dados
Análise de Algoritmos

- Considerações
  - O tempo de execução de cada operação primitiva depende do hardware e do software
    - Mas de todo modo é constante
  - Hipótese
    - O tempo de cada operação primitiva é praticamente o mesmo

Estrutura de Dados
Análise de Algoritmos

- Operações primitivas
  - Atribuição: a = x
  - Operação aritmética: a+1
  - Comparação de números: a>b
  - Indexar vetor: a[i]
  - Retorno de método: return x;
    - Custo unitário: custo(primitiva) = 1

      ou
    - Custo constante: custo(primitiva) = c

Estrutura de Dados
Análise de Algoritmos

- Custo das operações
  - Instruções consecutivas

```
Cmd1;
Cmd2;
```

custo(Cmd1) + custo(Cmd2)

  - If then else

```
if (teste) {
   ... // CustoIf
} else {
   ... // CustoElse
}
```

custo(teste) +
max[custo(if),custo(else)]

Estrutura de Dados
Análise de Algoritmos

- Custo de outras operações
  - Laço

```
for (...) {
    ... // CustoFor
}
```

Número de Iterações

n * custoFor

  - Aninhamento de Laços
    - tempo do laço interno x tempo do laço externo
  - Recursão
    - Mais complexo
    - Serão vistos três métodos
      - Substituição, iterativo e mestre

Estrutura de Dados
Análise de Algoritmos

- Exemplo 1
  - Determinar as primitivas do trecho abaixo

```
public int max(int x, int y) {
    if (x > y)
        return x;
    else return y;
}
```

custo = c1 +(c2 ou c3)

Estrutura de Dados
Análise de Algoritmos

- Exemplo 1
  - Determinar as primitivas do trecho abaixo

```
public int max(int x, int y) {
    if (x > y) c1
        return x; c2
    else return y; c3
}
```

custo = c1 +(c2 ou c3)

Estrutura de Dados
Análise de Algoritmos

- Exemplo 2
  - Determinar as primitivas do trecho abaixo

```
long potencia(int n) {
    long r = 1;
    for (int i=1; i<=n; i++)
        r=2*r;
    return r;
}
```

Estrutura de Dados
Análise de Algoritmos

- Exemplo 2
  - Determinar as primitivas do trecho abaixo

```
long potencia(int n) {
    long r = 1;        c1
    for (int i=1; i<=n; i++)   c2   c3*(n+1)   c4*(n)
        r=2*r;         n*c5
    return r;          c6
}
```

Estrutura de Dados
Análise de Algoritmos

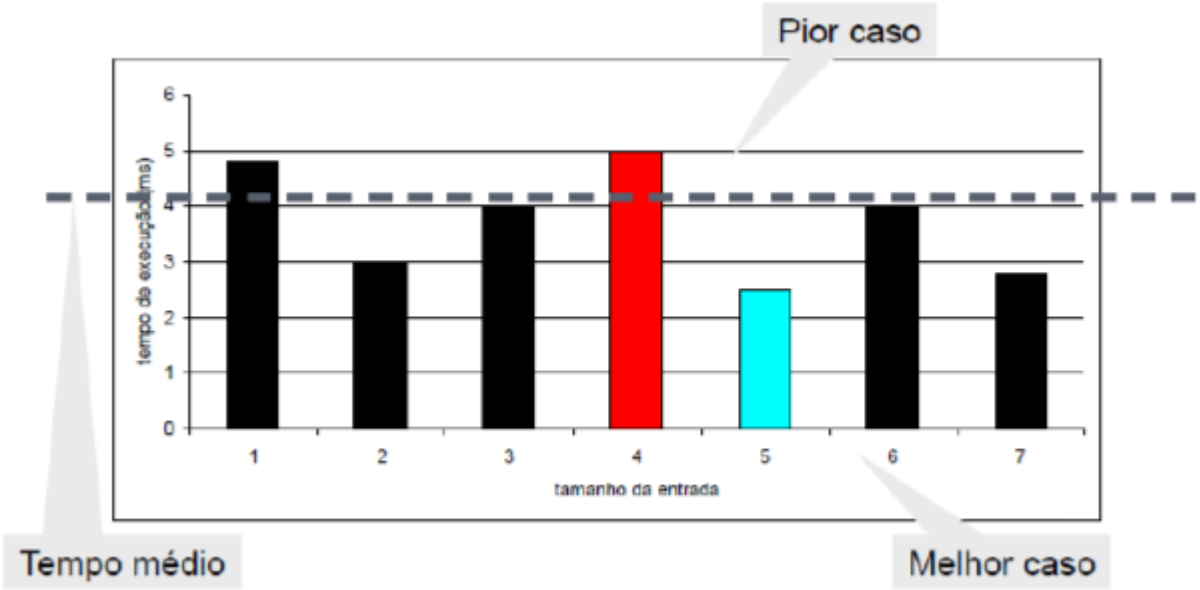
- Exemplo 2
  - Determinar as primitivas do trecho abaixo

```
long potencia(int n) {
    long r = 1;        c1
    for (int i=1; i<=n; i++)   c2   c3*(n+1)   c4*(n)
        r=2*r;         n*c5
    return r;          c6
}
```

custo = c1 + c2 + c3*(n+1) + c4*(n) + n*c5 + c6

Estrutura de Dados
Análise de Algoritmos

- Tipos de análises



Estrutura de Dados
Análise de Algoritmos

- Tipos de análises
  - Pior Caso
    - Mais comum
    - T(n) = tempo máximo para um algoritmo com qualquer entrada de tamanho n
    - Fácil de calcular (sem probabilidade)
  - Caso Médio
    - T(n) = tempo esperado sobre todas as entradas de tamanho n
    - Precisa de uma hipótese da distribuição estatística das entradas
  - Melhor Caso
    - Não acrescenta muita informação
    - Raramente ocorre na prática
    - Não é uma boa medida

Estrutura de Dados
Análise de Algoritmos

- Complexidade x Hardware

Computador 10^7 instruções/seg
Algoritmo de Ordenação
$50n\log10(n)$

Supercomputador 10^9 instruções/seg
Algoritmo de Ordenação
$2n^2$

Estrutura de Dados
Análise de Algoritmos

- Complexidade x Hardware – Análise
  - Tempo de ordenação para n = 106

10^7 instr/s
$50n\log_{10}(n)$

$50.10^6.\log_{10}10^6/10^7 \approx 30$ segundos

10^9 instr/s
$2n^2$

$2.10^{12}/10^9 = 2000$ segundos

  - É importante ter uma boa máquina boa
  - Também é importante um algoritmo eficiente

Estrutura de Dados
Análise Assintótica

- Introdução
  - Análise de algoritmos
    - Relembrando

```
public boolean XYZ(int n, ...) {
    ...
    ...
}
```

Algoritmos

Análises f(n) Matemática

    - Em geral, cada comando é executado em um tempo constante
      - O número de repetições de cada comando é que será relevante na complexidade

Estrutura de Dados
Análise Assintótica

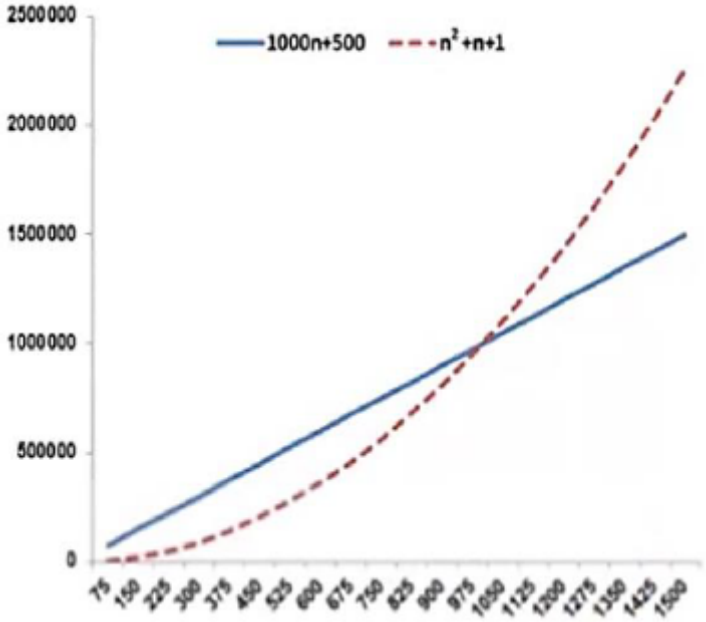
- Introdução
  - Ideia principal
    - Determinar como o algoritmo se comporta para valores muito grandes de entrada ($n \to \infty$)
      - Para tornar relevante apenas a ordem de crescimento do tempo de execução
  - Abstrações simplificadoras
    - Ignorar custo de operações primitivas
    - Ignorar constantes multiplicadoras
    - Ignorar valores de menor magnitude

Estrutura de Dados
Análise Assintótica

- Introdução
  - Exemplos de simplificação
    - $g(n) = 1000n+500 \rightarrow g(n)=n$
    - $h(n)=n^2+n+1 \rightarrow h(n)=n^2$
  - Deve ser observado
    - O tempo de execução
      - Verificar como ele cresce proporcionalmente a n
        - Polinomialmente, exponencialmente, logaritmicamente, ...

Estrutura de Dados
Análise Assintótica

- Introdução
  - Exemplo de análise
    - g(n) possui constantes maiores multiplicando seus termos
      - Mas existe um valor de n a partir do qual o resultado de h(n) é sempre maior do que g(n)



Estrutura de Dados
Análise Assintótica

- Introdução
  - Outros exemplos

Função Custo	Comportamento assintótico
$f(n) = 5n^3+200n^2+112$	$f(n) = n^3$
$f(n) = n^2+5n+2$	$f(n) = n^2$
$f(n) = 15n+2$	$f(n) = n$
$f(n) = 105$	$f(n) = 1$

  - Se a função não possui nenhum termo multiplicado por n, seu comportamento assintótico é constante (1)

Estrutura de Dados
Análise Assintótica

- Classe de funções
  - O tempo de execução de um algoritmo é uma função da entrada
    - Cada função tem uma taxa de crescimento
  - Uso de agrupamento de funções
    - Funções de mesmo grupo têm taxa de crescimento equivalente
      - Ex.: n^2, $2n^2$, $100n^2+n$ são todas funções que crescem na mesma velocidade
        - Quadrática

Estrutura de Dados
Análise Assintótica

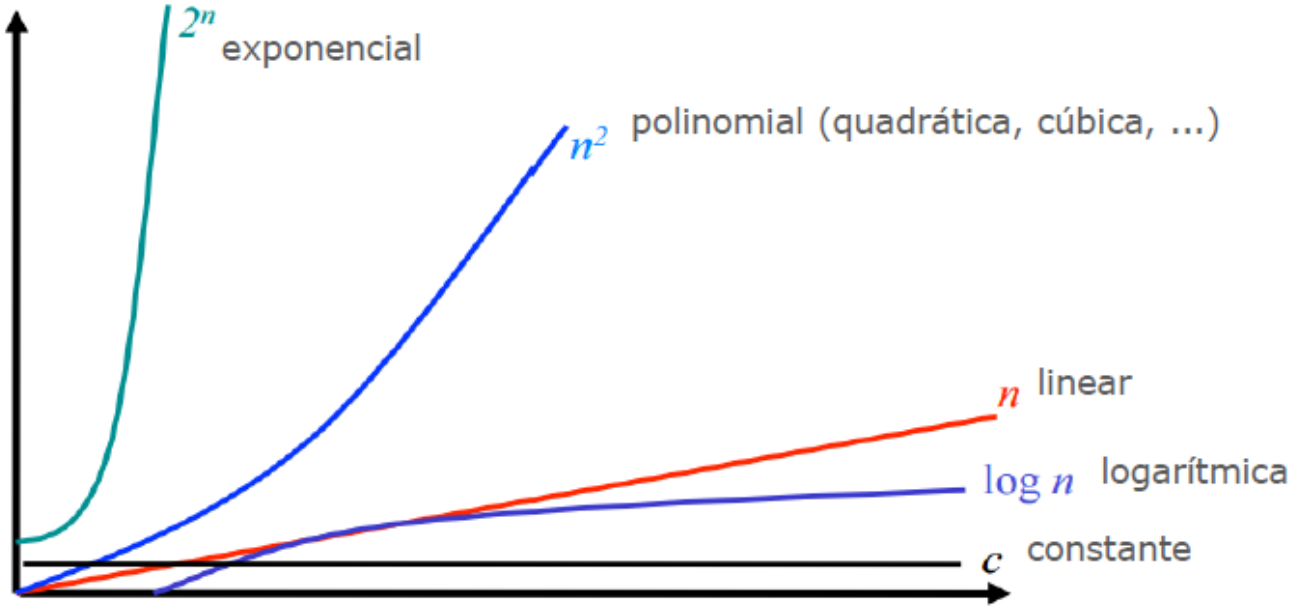
- Classe de funções
  - Exemplo
    - Classificação das funções abaixo por ordem de crescimento

a) n^5+132n^2 6
b) $3n+12$ 3
c) 8900 1
d) $45\log(n)$ 2
e) $45\log(n).n$ 4
f) $132n^2$ 5

MELHOR → PIOR

Estrutura de Dados
Análise Assintótica

- Classe de funções
  - Cada classe é caracterizada por uma família diferente de curva



Estrutura de Dados
Análise Assintótica

- Notação assintótica
  - O (o maiúsculo)
  - Ω (ômega maiúsculo)
  - Θ (theta)
  - o (o minúsculo)
  - ω (ômega minúsculo)
- Abusos da notação matemática

Conjunto de funções

$$2n + 1 = \Theta(n)$$

$\in$

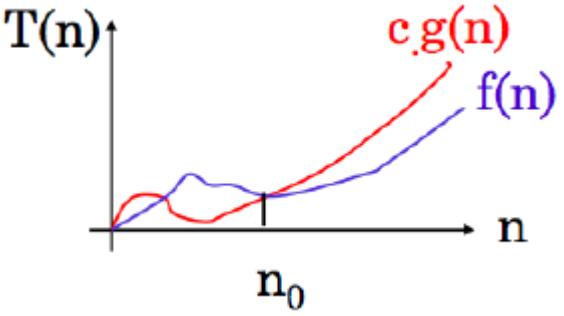
Estrutura de Dados
Análise Assintótica

- Notação O
  - Descreve o limite assintótico superior
    - Utilizada para analisar o pior caso do algoritmo
  - Exemplo: $O(n^2)$
    - Indica que o custo do algoritmo é, assintoticamente, menor ou igual a n^2
      - O custo do algoritmo é no máximo tão ruim quanto n^2
    - Notação mais conhecida e utilizada

Estrutura de Dados
Análise Assintótica

- Notação O

$$O(g(n)) = \{f(n) \mid \exists c>0,\ n_0>0,\ 0 \le f(n) \le c\ g(n)\ \forall n \ge n_0\}$$



  - Para todos os valores de n à direita do valor n_0
    - O resultado da função custo f(n) é sempre menor ou igual ao valor da função g(n) multiplicada por uma constante c
    - f(n) = 0(g(n))
      - "f(n) cresce no máximo tão rapidamente quanto g(n)"
      - g(n) é um limite superior assintótico para f(n)

Estrutura de Dados
Análise Assintótica

- Notação O
  - Exemplo
    - A função $f(n) = 3n^3+2n^2$ é $O(n^3)$

      $c = 6$

      $n \geq 0$

      $3n^3+2n^2 \leq 6n^3$

$$O(g(n)) = \{f(n) \mid \exists c>0,\ n_0>0,\ 0 \leq f(n) \leq c\ g(n)\ \forall n \geq n_0\}$$

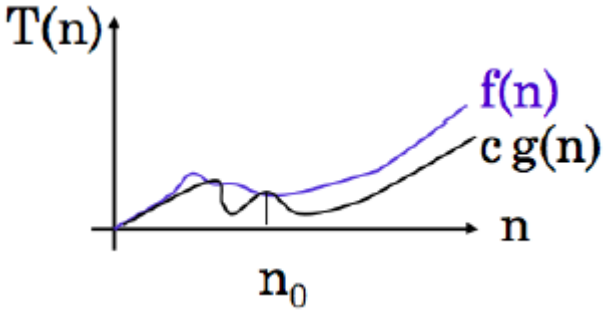
Estrutura de Dados
Análise Assintótica

- Notação Ω
  - Descreve o limite assintótico inferior
    - Utilizada para analisar o melhor caso do algoritmo
  - Exemplo: $\Omega(n^2)$
    - Indica que o custo do algoritmo é, assintoticamente, maior ou igual a n^2
      - O custo do algoritmo é no mínimo tão ruim quanto n^2

Estrutura de Dados
Análise Assintótica

- Notação Ω

$$\Omega(g(n)) = \{f(n) \mid \exists c>0,\ n_0>0,\ 0 \le cg(n) \le f(n)\ \forall n \ge n_0\}$$



  - Para todos os valores de n à direita do valor n_0
    - O resultado da função custo f(n) é sempre maior ou igual ao valor da função g(n) multiplicada por uma constante c
    - $f(n) = \Omega(g(n))$
      - "f(n) cresce no mínimo tão lentamente quanto g(n)"
      - g(n) é um limite inferior assintótico para f(n)

Estrutura de Dados
Análise Assintótica

- Notação Ω
  - Exemplo
    - A função $f(n) = 3n^3+2n^2$ é $\Omega(n^3)$

      $c = 1$

      $n \geq 0$

      $3n^3+2n^2 \leq 1n^3$

$$\Omega(g(n)) = \{f(n) \mid \exists c>0,\ n_0>0,\ 0 \leq cg(n) \leq f(n)\ \forall n \geq n_0\}$$

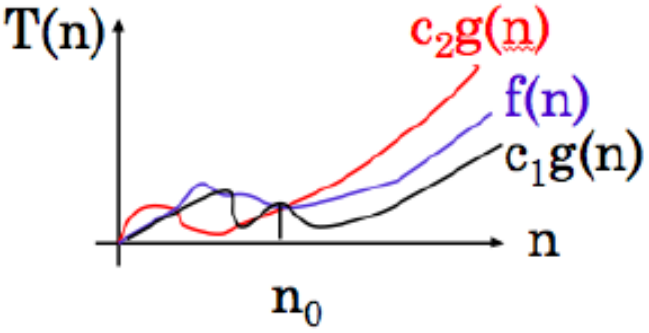
Estrutura de Dados
Análise Assintótica

- Notação Θ
  - Descreve o limite assintótico restrito
    - Utilizada para analisar os limites inferior e superior do algoritmo
    - Analisa o caso médio
  - Exemplo: $\Theta(n^2)$
    - Indica que o custo do algoritmo é, assintoticamente, igual a n^2
      - O custo do algoritmo é n^2, dentro de um fator constante abaixo e acima

Estrutura de Dados
Análise Assintótica

- Notação Θ

$$\Theta(g(n)) = \{f(n) \mid \exists c_1>0,\ c_2>0,\ n_0>0,\ 0 \le c_1 g(n) \le f(n) \le c_2 g(n)\ \ \forall n \ge n_0\}$$



  - Para todos os valores de n à direita do valor n_0
    - O resultado da função custo f(n) é sempre igual ao valor da função g(n) multiplicada por uma constantes c_1 e c_2
    - f(n) = Θ(g(n))
      - "f(n) cresce tão rapidamente quanto g(n)"
      - g(n) é um limite assintótico restrito para f(n)

Estrutura de Dados
Análise Assintótica

- Notação Θ
  - Exemplo 1
    - A função $f(n) = 7n^4+5n^2+10$ é $\Theta(n^4)$

      $c_1 = 7$

      $c_2 = 22$

      $n \geq 1$
    - $7n^4 \leq 7n^4 + 5n^2 + 10 \leq 22n^4$

$$\Theta(g(n)) = \{f(n) \mid \exists c_1>0,\ c_2>0,\ n_0>0,\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\ \ \forall n \geq n_0\}$$

Estrutura de Dados
Análise Assintótica

- Notação Θ
  - Exemplo 2
    - A função $f(n) = (1/2)n^2 - 3n$ é $\Theta(n^2)$

      $c_1 = 1/14$

      $c_2 = 1/2$

      $n \geq 7$

      $(1/14)n^2 \leq (1/2)n^2 - 3n \leq (1/2)n^2$

$$\Theta(g(n)) = \{f(n) \mid \exists c_1>0,\ c_2>0,\ n_0>0,\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\ \ \forall n \geq n_0\}$$

Estrutura de Dados
Análise Assintótica

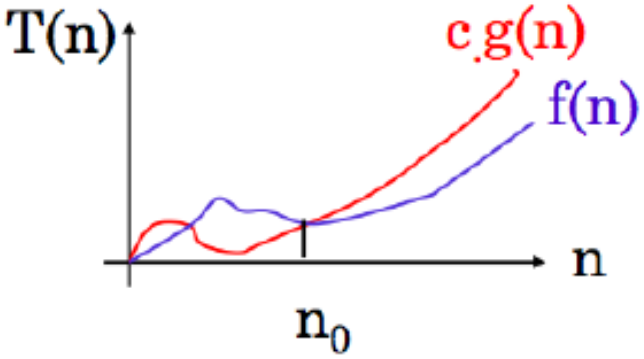
- Notação o
  - Descreve o limite assintótico superior não restrito
  - Exemplo: $o(n^2)$
    - Indica que o custo do algoritmo é, assintoticamente, menor que n^2

- Notação ω
  - Descreve o limite assintótico inferior não restrito
  - Exemplo: $\omega(n^2)$
    - Indica que o custo do algoritmo é, assintoticamente, maior que n^2

Estrutura de Dados
Análise Assintótica

- Notação o

$$o(g(n)) = \{f(n) \mid \text{para qualquer } c>0, \text{ existe } n_0>0, \; 0 \le f(n) < c\, g(n) \text{ para todo } n \ge n_0\}$$

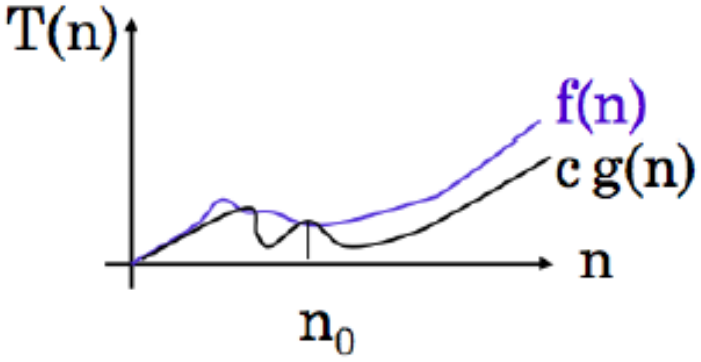


  - Para todos os valores de n à direita do valor n_0
    - O resultado da função custo f(n) é sempre menor que o valor da função g(n) multiplicada por uma constante c
    - f(n) = o(g(n))
      - "f(n) cresce mais lentamente que g(n)"
      - g(n) é um limite superior assintótico não restrito para f(n)

Estrutura de Dados
Análise Assintótica

- Notação ω

$$\omega(g(n)) = \{f(n) \mid \text{para qualquer } c>0, \text{ existe } n_0>0, \; 0 \le cg(n) < f(n) \text{ para todo } n \ge n_0\}$$



  - Para todos os valores de n à direita do valor n_0
    - O resultado da função custo f(n) é sempre maior que o valor da função g(n) multiplicada por uma constante c
    - f(n) = ω(g(n))
      - "f(n) cresce no mínimo mais rapidamente que g(n)"
      - g(n) é um limite inferior assintótico não restrito para f(n)

Estrutura de Dados
Análise Assintótica

- Propriedades

Transitividade:

Se $f(n) \in O(g(n))$ e $g(n) \in O(h(n))$, então $f(n) \in O(h(n))$.

Se $f(n) \in \Omega(g(n))$ e $g(n) \in \Omega(h(n))$, então $f(n) \in \Omega(h(n))$.

Se $f(n) \in \Theta(g(n))$ e $g(n) \in \Theta(h(n))$, então $f(n) \in \Theta(h(n))$.

Se $f(n) \in o(g(n))$ e $g(n) \in o(h(n))$, então $f(n) \in o(h(n))$.

Se $f(n) \in \omega(g(n))$ e $g(n) \in \omega(h(n))$, então $f(n) \in \omega(h(n))$.

Reflexividade:

$f(n) \in O(f(n))$.

$f(n) \in \Omega(f(n))$.

$f(n) \in \Theta(f(n))$.

Simetria:

$f(n) \in \Theta(g(n))$ se, e somente se, $g(n) \in \Theta(f(n))$.

Simetria Transposta:

$f(n) \in O(g(n))$ se, e somente se, $g(n) \in \Omega(f(n))$.

$f(n) \in o(g(n))$ se, e somente se, $g(n) \in \omega(f(n))$.

Estrutura de Dados
Análise Assintótica

- Resumindo

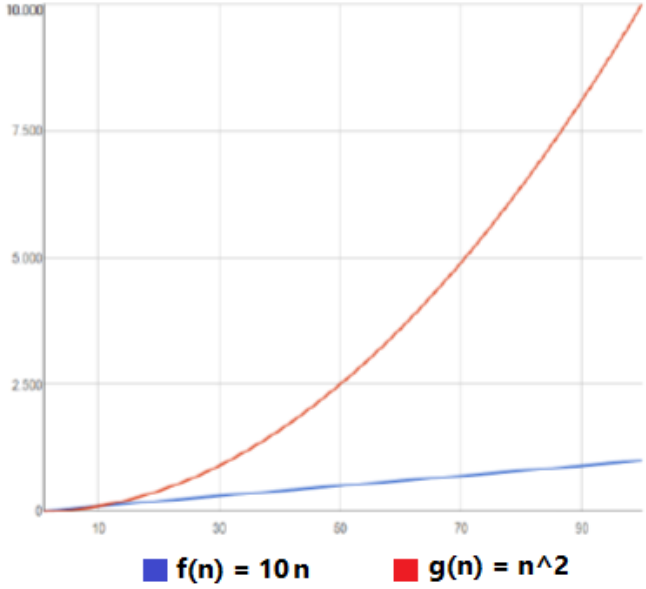
$$O \approx \leq$$
$$\Omega \approx \geq$$
$$\Theta \approx =$$
$$o \approx <$$
$$\omega \approx >$$

Estrutura de Dados
Análise Assintótica

- Observação
  - Se $f(n) = O(n)$ e $g(n) = O(n^2)$, $f(n)$ sempre crescerá mais lentamente que $g(n)$?

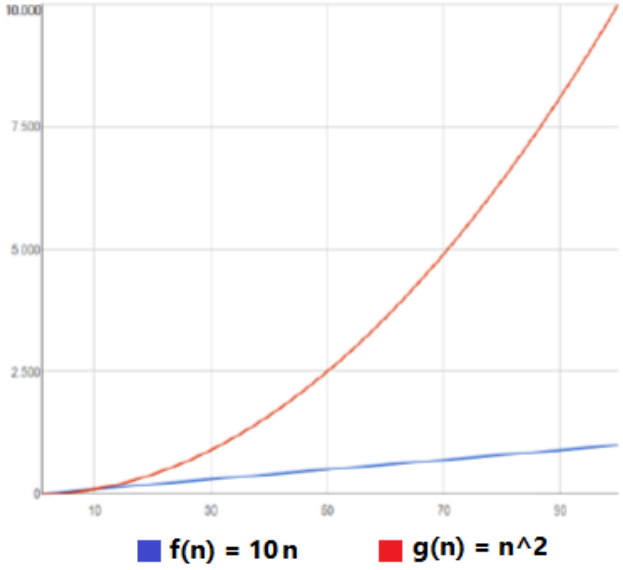
Estrutura de Dados
Análise Assintótica

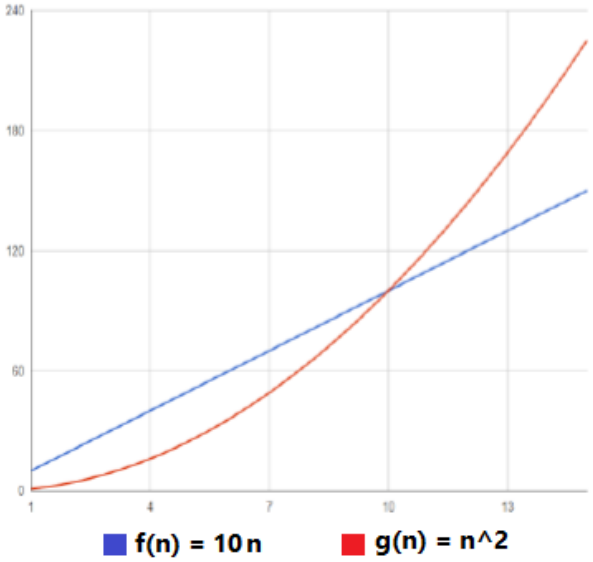
- Observação
  - Se $f(n) = O(n)$ e $g(n) = O(n^2)$, $f(n)$ sempre crescerá mais lentamente que $g(n)$?



Estrutura de Dados
Análise Assintótica

- Observação
  - Se f(n) = O(n) e g(n) = O(n^2), f(n) sempre crescerá mais lentamente que g(n)?





Estrutura de Dados
Análise Assintótica

- Classes de problemas
  - O(1): ordem constante
    - Instruções executadas um numero fixo de vezes
      - Não depende do tamanho dos dados de entrada
  - O(log(n)): ordem logarítmica
    - Típica de algoritmos que resolvem um problema transformando-o em problemas menores

Estrutura de Dados
Análise Assintótica

- Classes de problemas
  - O(n): ordem linear
    - Em geral, uma certa quantidade de operações é realizada sobre cada um dos elementos de entrada
  - O(n.log(n)): ordem log linear
    - Típica de algoritmos que trabalham com particionamento de dados
      - Esses algoritmos resolvem um problema transformando-o em problemas menores, que são resolvidos de forma independente e depois unidos

Estrutura de Dados
Análise Assintótica

- Classes de problemas
  - $O(n^2)$: ordem quadrática
    - Normalmente ocorre quando os dados são processados aos pares
      - Caracterizada pela presença de 2 estruturas de repetição aninhadas
  - $O(n^3)$: ordem cúbica
    - Caracterizada pela presença de 3 estruturas de repetição aninhadas

Estrutura de Dados
Análise Assintótica

- Classes de problemas
  - $O(2^n)$: ordem exponencial
    - Geralmente ocorre quando se usa uma solução de "força bruta"
  - • Algoritmos não úteis do ponto de vista prático
  - $O(n!)$: ordem fatorial
    - Geralmente ocorre quando se usa uma solução de "força bruta"
      - Algoritmos não úteis do ponto de vista prático
      - Comportamento muito pior que o exponencial

Estrutura de Dados
Análise Assintótica

- Classes de problemas
  - Comparação sobre o tempo de execução
    - Considerando um computador capaz de executar 1 milhão de operações por segundo

f(*n*)	*n* = 10	*n* = 20	*n* = 50	*n* = 100
log n	0,000003 seg.	0,000004 seg.	0,000006 seg.	0,000007 seg.
n	0,00001 seg.	0,00002 seg.	0,00005 seg.	0,00010 seg.
n . log n	0,00003 seg.	0,00009 seg.	0,00028 seg.	0,00066 seg.
n^2	0,0001 seg.	0,0004 seg.	0,0025 seg.	0,0100 seg.
n^3	0,001 seg.	0,008 seg.	0,125 seg.	1 seg.
2^n	0,001 seg.	1,05 seg.	17,9 min.	4,02E+14 séc.
n!	3,63 seg.	771,46 séc.	8,41E+16 séc.	2,96E+142 séc.

Estrutura de Dados
Análise Assintótica

- Limites superior e inferior
  - Multiplicação de duas matrizes nxn
    - Método trivial: algoritmo de complexidade $O(n^3)$
      - O limite superior de um problema pode mudar se alguém descobrir um outro algoritmo melhor
        - Strassen (1969): $O(n^{2.807})$
        - Coppersmith e Winograd (1990): $O(n^{2.376})$
        - Stothers (2010): $O(n^{2.373})$
        - Willians (2011): $O(n^{2.372})$

Estrutura de Dados
Análise Assintótica

- Limites superior e inferior
  - Apenas ler os elementos das duas matrizes de entrada ou produzir os elementos da matriz produto
    - Complexidade é $O(n^2)$
      - O limite inferior trivial é $\Omega(n^2)$
  - Um algoritmo com complexidade igual ao limite inferior do problema é ótimo

Estrutura de Dados
Análise de Algoritmos Recursivos

- Recorrência
  - Todo algoritmo recursivo tem uma recorrência associada
  - Recorrência é uma equação ou inequação que descreve uma função em termos dela mesma
    - Só que com valores menores
    - Exemplo
      - $T(n) = 2T(n/2) + 0(n)$

Estrutura de Dados
Análise de Algoritmos Recursivos

- Recorrência
  - Equação de recorrência do fatorial recursivo

```
int fatorial (int n) {
    if (n == 0)
        return 1;
    else
        return n * fatorial(n-1);
}
```

T(n)

O(1)

T(n-1)

  - $T(n) = T(n-1) + O(1)$

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Substituição
  - Iterativo
  - Mestre

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Substituição
    - Dois passos
      - Estimar (“chutar”) uma solução
      - Usar indução matemática para mostrar que o “chute” é válido
    - Pode ser usada para estabelecer limites superiores e inferiores para a recorrência
    - Possíveis problemas
      - Estimação da solução
      - Cuidados na aplicação da indução

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Substituição
    - Exemplo
      - $T(n) = 2T(n/2) + n$
      - Chute: $T(n) = O(n \lg n)$
      - Deve-se provar portanto que: $T(n) \leq c.n \lg n$

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Substituição
    - Exemplo
      - Passo indutivo
        - Mostrar que: se a a solução é verdadeira para todo m < n especialmente para n/2 ela é válida para T(n)
        - Isso é feito substituindo-se T(n/2) na equação de recorrência

Equação de recorrência $T(n) = 2T(n/2) + n$

"Chute" $T(n) \leq c.n \lg n$

$T(n/2) \leq cn/2 \lg(n/2)$

$T(n) \leq 2(cn/2 \lg(n/2)) + n$

$T(n) \leq cn \lg(n/2) + n$

$= cn \lg n - cn \lg 2 + n$

$= cn \lg n - cn + n$

$T(n) \leq cn \lg n$ para $c \geq 1$

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Substituição
    - Exemplo
      - Provando que a solução é válida para o caso base
        - $T(1) = 2(T(1/2)) + 1$. Como $T(1)$ é o menor valor, $T(1/2) = 0$
        - $T(1) = 1$
        - $c.n.\ln n = 1.1.\ln 1$
          $= 1.1.0$
          $= 0$
        - $T(n) \leq c.n.\ln n \rightarrow T(1) \leq c.1.\lg 1 \rightarrow 1 \leq 0$
          - Solução não é válida para o caso base

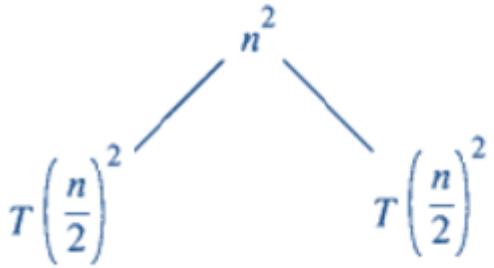
Estrutura de Dados
Análise de Algoritmos Recursivos

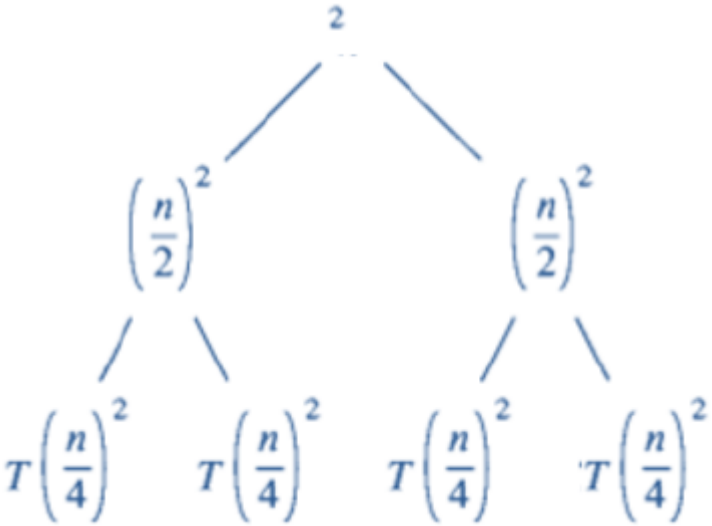
- Métodos de análise de complexidade
  - Substituição
    - Exemplo
      - Provando que a solução é válida para o caso base
        - Para resolver esse problema, basta lembrar que a notação assintótica exige que $T(n) \leq c.n \lg n, \forall n \geq n_0$
        - Portanto, basta escolher outro caso base, como T(2). Pela equação de recorrência, se T(1) = 1 então
        - $T(2) = 2(T(2/2)) + 2 \leq c.n.\lg n = c.2.\lg 2$
        - $2(T(1)) + 2 \leq c.2.1$
        - $2.1 + 2 \leq 2c$
        - $4 \leq 2c$, para $c \geq 2$
        - Logo, mostramos que $T(2) \leq c.2 \lg 2$ é válida para $c \geq 2$
        - Portando, a solução $T(n) = O(n.\lg(n))$ funciona

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Iterativo
    - Visualização conveniente da expansão de uma equação de recorrência

$T(n) = 2T(n/2) + n^2$





Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Iterativo
    - Procedimento
      - 1. Desenhe a árvore de recursão com base na equação de recorrência (T)
      - 2. Encontre a maior altura (h) até o caso base
      - 3. Some os custos de cada nível
      - 4. Analise o relacionamento entre os custos de cada nível
      - 5. Some os custos de todos os níveis com base na recorrência encontrada em 1

Estrutura de Dados
Análise de Algoritmos Recursivos

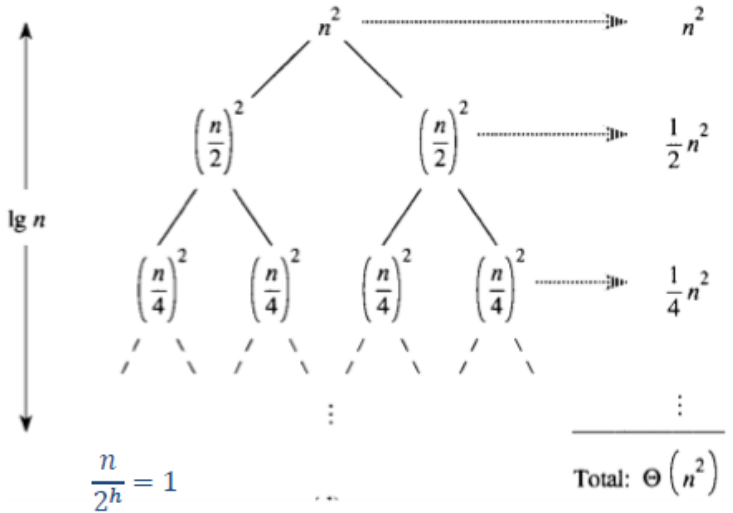
- Métodos de análise de complexidade
  - Método Iterativo
    - Exemplo 1

```
exemplo(A)

        if n == 1
                return A[1]

        exemplo(A[1..n/2])
        exemplo(A[(n/2)+1..n])

        for i = 0 to N
                for j = 0 to N
                        // do something
```

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Iterativo
    - Exemplo 1

$$T(n) = 2T(n/2) + n^2$$



$$\frac{n}{2^h} = 1$$
$$h = \log n$$

$$S_n = n^2(1 + 1/2 + 1/4 + \ldots + 1/2^h)$$

Série Geométrica Decrescente
n grande → PG infinita

$q = 1/2$

$$S_\infty = \frac{a_1}{1-q}, |q| < 1$$

$$\begin{aligned} S_n &= a1/(1-q) \\ &= n^2/(1-1/2) \\ &= n^2/(1/2) \\ &= 2n^2 \\ &= \Theta(n^2) \end{aligned}$$

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Iterativo
    - Exemplo 2

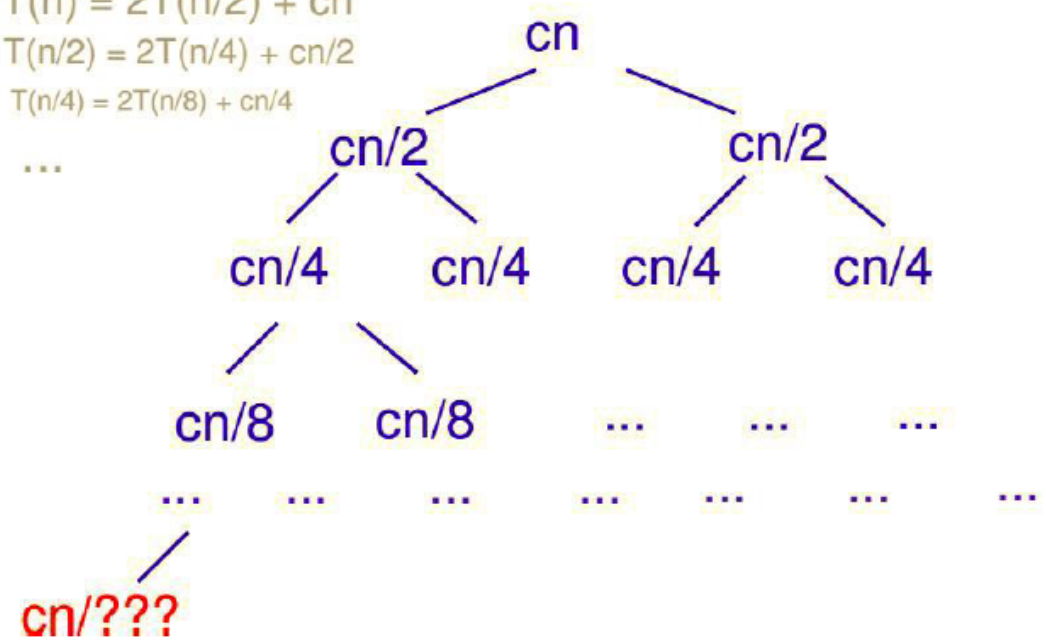
      $T(n) = 2T(n/2)+n$

$T(n) = 2T(n/2) + cn$

$T(n/2) = 2T(n/4) + cn/2$

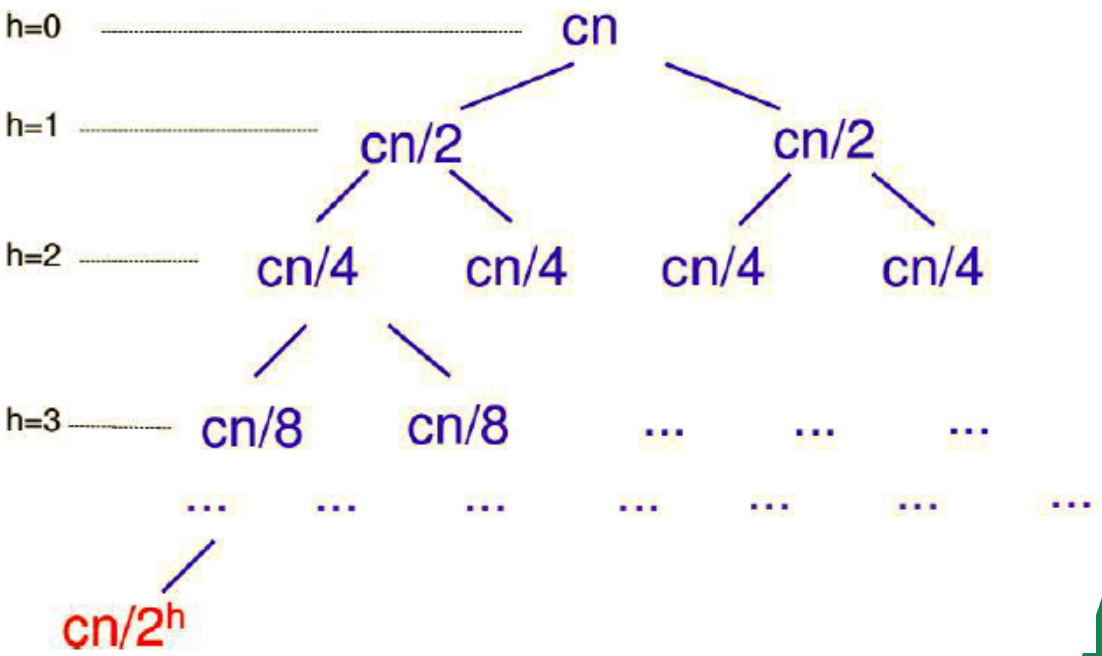
$T(n/4) = 2T(n/8) + cn/4$

...



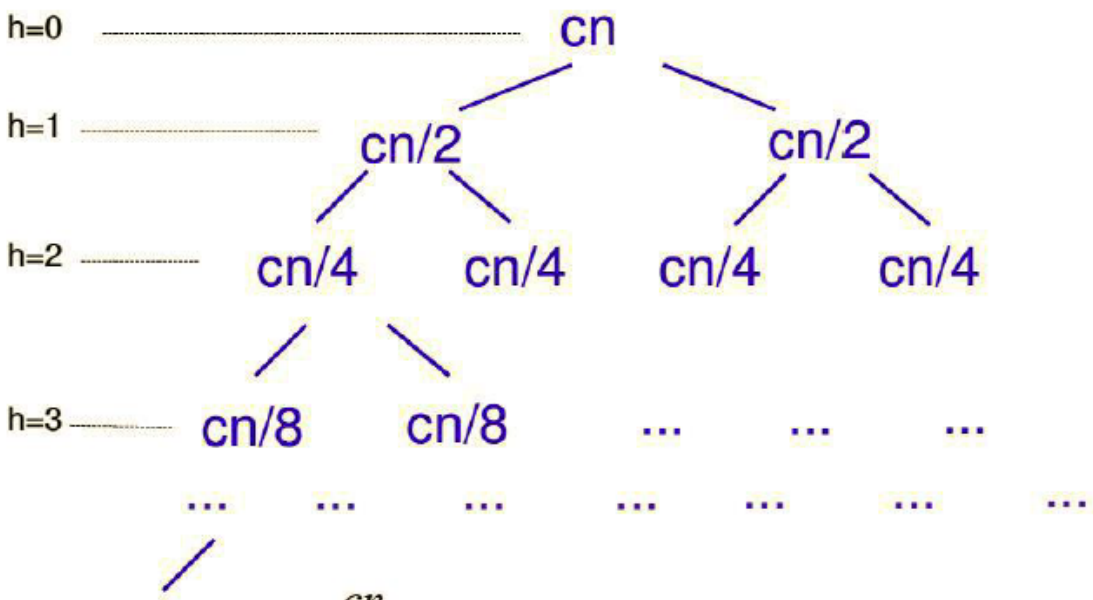
Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Iterativo
    - Exemplo 2
      $T(n) = 2T(n/2)+n$



Estrutura de Dados
Análise de Algoritmos Recursivos

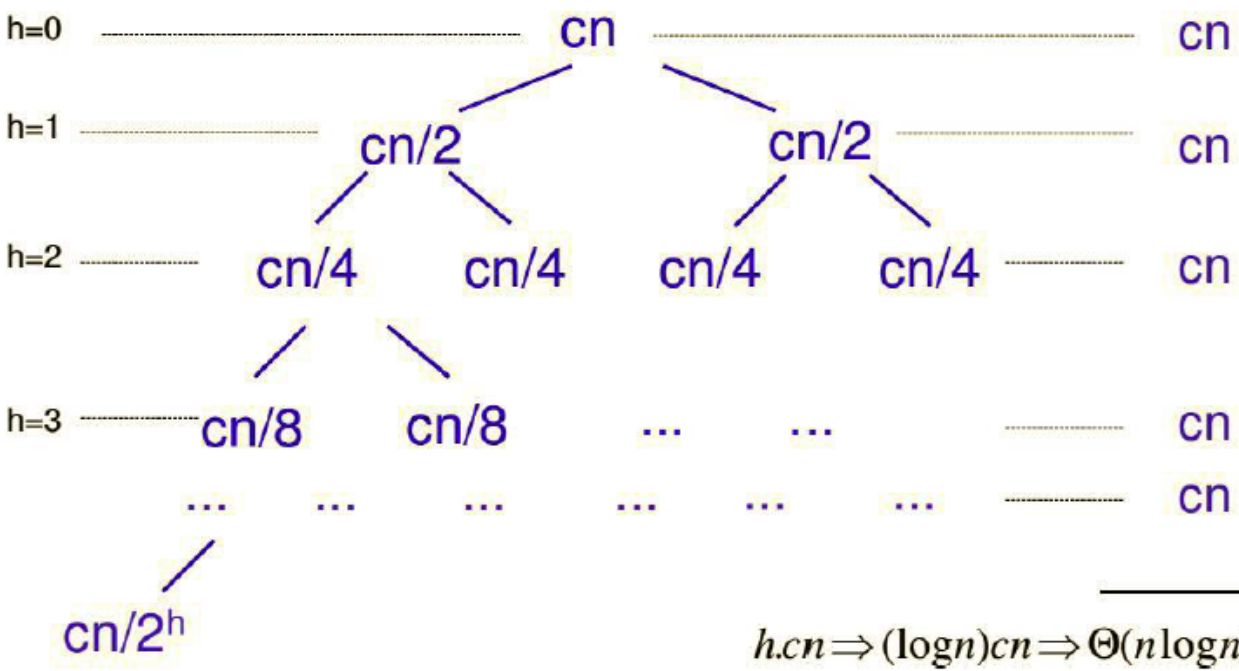
- Métodos de análise de complexidade
  - Método Iterativo
    - Exemplo 2
      T(n) = 2T(n/2)+n



$cn/2^h = k \Rightarrow \frac{cn}{2^h} = k \Rightarrow 2^h = cn/k \Rightarrow h = \log(cn/k)$ **h = log n**

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Iterativo
    - Exemplo 2
      T(n) = 2T(n/2)+n



$$h.cn \Rightarrow (\log n)cn \Rightarrow \Theta(n \log n)$$

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Mestre
    - “Livro de receitas” para boa parte das recorrências deste tipo
      - $T(n) = aT(n/b) + f(n)$
        - a= número de subproblemas (a >= 1)
        - b= tamanho do subproblema (b > 1)
        - f(n) não negativa
    - Cada um dos a subproblemas tem custo T(n/b)
    - O tempo de recombinar os subproblemas é f(n)

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Mestre
    - Dada a função T(n) = aT(n/b) + f(n)
    - Casos

$$f(n) < n^{\log_b a} \xrightarrow{\text{Caso 1}} T(n) = \Theta(n^{\log_b a})$$

$$f(n) = n^{\log_b a} \xrightarrow{\text{Caso 2}} T(n) = \Theta(n^{\log_b a} \log_b n) = \Theta(f(n).\log_b n)$$

$$f(n) > n^{\log_b a} \xrightarrow{\text{Caso 3}} T(n) = \Theta(f(n))$$

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Mestre
    - Procedimentos
      - Identificar a, b, f(n), $\log_b a$
      - Verificar as condições sobre $a \geq 1$, $b > 1$ e f(n)
      - Comparar f(n)com $n^{\log_b a}$ para identificar o caso
      - Confirmar a condição do caso
      - Encontrar o Θ

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Mestre
    - Caso 01 $f(n) < n^{\log_b a} \Rightarrow T(n) = \Theta(n^{\log_b a})$

$$T(n) = aT(n/b) + f(n)$$

    - Exemplo $T(n) = 8T\left(\frac{n}{2}\right) + 1000n^2$ $\qquad f(n) = 1000n^2$

$$n^{\log_2 8}$$

$$\boxed{1000n^2 < n^{\log_2 8}}$$

$$T(n) = \Theta\left(n^3\right)$$

Lembrete:
$\log_b a = x \Leftrightarrow a = b^x$

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Mestre
    - Caso 02 $f(n) = n^{\log_b a}$ $\Rightarrow$ $T(n) = \Theta(n^{\log_b a} \log_b n)$ $= \Theta(f(n).\log_b n)$

$$T(n) = aT(n/b) + f(n)$$

    - Exemplo $T(n) = 2T\left(\frac{n}{2}\right) + 10n$ f(n) = 10n

$n^{\log_2 2}$

$$\boxed{10n = n^{\log_2 2}}$$

10n

$$T(n) = \Theta(n \log n)$$

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Mestre
    - Caso 03 $f(n) > n^{\log_b a} \Rightarrow T(n) = \Theta(f(n))$

$$T(n) = aT(n/b) + f(n)$$

    - Exemplo $T(n) = 2T\left(\frac{n}{2}\right) + n^2$ $\qquad f(n) = n^2$

$$n^{\log_2 2}$$

$$\boxed{n^2 > n^{\log_2 2}}$$

$n^2 >$

$$T(n) = \Theta\left(n^2\right)$$

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Mestre
    - Casos onde não é possível a aplicação

$$T(n) = 2^n T\left(\frac{n}{2}\right) + n^n$$

a=2^n não é constante

$$T(n) = 0.5T\left(\frac{n}{2}\right) + \frac{1}{n}$$

a=0.5 < 1

$$T(n) = 64T\left(\frac{n}{8}\right) - n^2 \log n$$

f(n) não é não-negativa

Estrutura de Dados
Análise de Algoritmos Recursivos

- Métodos de análise de complexidade
  - Método Mestre
    - Casos onde não é possível a aplicação
      - Quando f(n) não é maior ou menor polinomialmente com relação a $n \log_b a$
      - Exemplo

$$T(n) = 2T(n/2) + n \lg n$$

$a = 2$

$b = 2$

$f(n) = n \lg n$

$n \lg n > n^{\log_b a}$

Mas não é maior polinomialmente

Como Verificar?

$$\frac{f(n)}{n^{\log_b a}} = \frac{(n \lg n)}{n} = \lg n$$

Assintoticamente menor que n^{ε} pra qualquer constante positiva ε